

레이다 SW 학습 과제 3

표적 궤적 모의

Target Trajectory Simulation

Janghwan Kim (Harley)

표적 궤적 모의 TargetSim

Janghwan Kim (Harley)

01

표적 구조체 설계

상태 벡터와 좌표계 정의
기동을 $G \cdot \text{축} \cdot \text{구간}$ 으로 환원
구조체 관계와 실제 헤더

02

표적 궤적 모의 구현

운동학 모델과 회전행렬 전개
동체 속도 $\rightarrow$ ECEF 속도 (추가 고민)
중점법 적분과 방어적 구현

03

시뮬레이션 결과 및 검증

명세 조건 60s 결과
해석해 대조 · 불변량 보존 · 수렴 차수
기동 시연과 운용 화면

과제 요구사항과 발표 구성

「과제 3 내용」의 요구사항을 네 덩어리로 묶고, 각각을 어느 장에서 다루는지 먼저 짚겠습니다.

1 표적 구조체 정의

표적 최대 10, 표적별 기동 최대 30.
위치는 위도 · 경도 · 고도,
속도는 동체 속도 $V_{heading}$ 한 개.
기동은 G · 축 · 시작 · 종료.

01 장

2 표적 상태 갱신

위치와 속도를 ECEF 에서 갱신.
추가 고민 — $V_{heading}$ 을 ECEF
 V_x, V_y, V_z 로 변환하는 방법.

02 장

3 시뮬레이션 조건

$60\text{ s} \cdot 0.1\text{ s}$ 간격.
플랫폼 1 + 대함 1 + 대공 1.
제공받은 좌표변환 코드 사용.

03 장

4 결과 제시

궤적을 LLA 로 출력하고 그림으로.
구조체 · 구현 방법 · 결과를
코드와 함께 설명.

03 장

좌표변환과 행렬 연산은 제공받은 코드를 한 줄도 고치지 않고 그대로 사용했습니다. 새로 작성한 것은 TargetSim.c 하나입니다.
발표에 나오는 숫자는 모두 그 Core 를 실제로 빌드해 돌린 결과이며, 원본 CSV 를 함께 제출합니다.

들어가며

표적 모의가 들어가는 자리

레이다 데이터처리 소프트웨어를 만들려면, 그 앞단이 내보내는 것과 같은 규격의 입력이 필요합니다.

레이다 데이터처리 체인에서 모의가 대신하는 구간



모의를 쓰는 이유

	실제 해상 시험	표적 모의
반복성	같은 상황 재현 불가	몇 번이든 동일
참값(truth)	기준 장비가 또 필요, 그 값도 오차	입력값이 곧 참값
조건 제어	날씨 · 구역 · 표적 가용성에 종속	G · 침로 · 고도 임의
소요	함정 · 항공기 · 인력 · 기간	60 s 궤적이 5 ms

모의가 만족해야 할 조건

- 규격 일치** 데이터처리 입력과 같은 형식
시각마다 위 · 경 · 고도 — 본 과제는 0.1 s 간격 601 표본
- 물리적 타당성** 불변량이 보존될 것
등속 수평 비행이면 속력과 고도가 변하지 않아야 한다
- 수치적 재현성** 정해진 차수로 수렴할 것
갱신 간격 Δt 를 절반으로 줄이면 오차가 1/4 로 (2차)

소프트웨어 구성과 인터페이스

연산과 화면을 분리해, 계산 엔진만 따로 빌드해 검증할 수 있게 했습니다.



01

표적 구조체 설계

Target State and Structure Design

- 표적 하나의 상태를 어떤 변수로 잡을 것인가
- "기동" 을 코드가 다룰 수 있는 네 개의 값으로 환원하기
- 설정 (입력) 과 상태 (출력) 를 분리한 이유

01. 표적 구조체 설계

표적 상태 벡터의 정의

위치 3 + 속도 3 + 자세각 3, 모두 아홉 개 성분으로 표적 하나의 상태가 결정됩니다.

표적 하나의 상태 벡터

$$\mathbf{x}(t) = [\mathbf{p}^e(t)^T \quad \mathbf{v}^e(t)^T \quad \boldsymbol{\Theta}(t)^T]^T \in \mathbb{R}^9$$

$\mathbf{p}^e = [x \ y \ z]^T$	ECEF 위치	m	3
$\mathbf{v}^e = [v_x \ v_y \ v_z]^T$	ECEF 속도	m/s	3
$\boldsymbol{\Theta} = [\phi \ \theta \ \psi]^T$	자세각 (Roll, Pitch, Yaw)	rad	3

위치와 속도는 ECEF 에서 갱신하고, LLA 는 출력·다음 스텝용으로 함께 보관한다.

명세가 주는 입력 → 초기 상태

위도 · 경도 · 고도
 $(\varphi_0, \lambda_0, h_0)$
 $\xrightarrow{\text{blue}} \mathbf{p}_0 = f_{lla \rightarrow ecef}$

자세각
 $(\phi_0, \theta_0, \psi_0)$
 $\xrightarrow{\text{orange}} \boldsymbol{\Theta}_0$ (그대로)

동체 속력 1개
 V
 $\xrightarrow{\text{green}} \mathbf{v}_0^e = \mathbf{C}_n^e \mathbf{C}_b^n [V \ 0 \ 0]^T$
→ 세 번째 줄이 이 과제의 추가 고민 항목

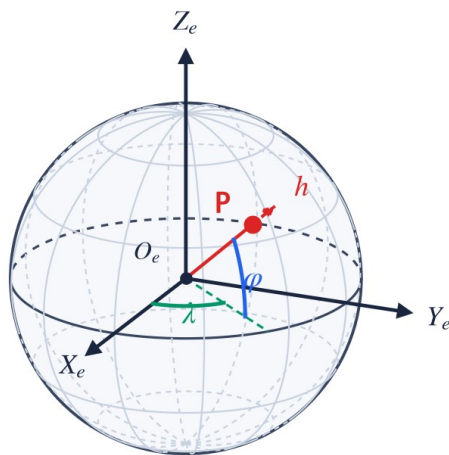
01. 표적 구조체 설계

쓰는 좌표계 — ECEF · LLA · NED · Body

Part 2 에서 정리한 좌표계 중 표적 모의에 실제로 필요한 것은 이 넷입니다.

ECEF — 지구중심 지구고정 좌표계

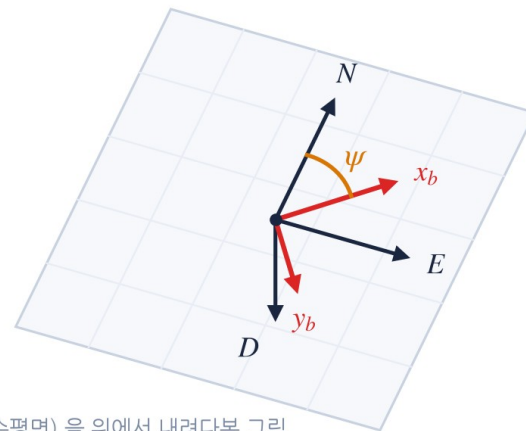
원점 지구중심, Z_e 자전축, X_e 본초자오선. 지구와 함께 돈다.



같은 점 P를 각도로 적은 것이 LLA (φ, λ, h) — 입·출력 전용

NED · Body — 국지 좌표계

원점은 표적 자신. N 북 · E 동 · D 지심방향, x_b 는 기수 방향.



국지 접평면 (수평면) 을 위에서 내려다본 그림

수평 비행이면 $\phi = \theta = 0$ 이라 x_b 와 N 사이 각이 곧 ψ 다



01. 표적 구조체 설계

자세각 정의와 회전 순서

Roll · Pitch · Yaw 가 각각 어느 축 둘레의 회전인지 , 그리고 곱하는 순서를 고정합니다 .



동체 → NED 회전행렬은 세 회전의 곱이다

$$C_b^n(\phi, \theta, \psi) = \mathbf{R}_z(\psi) \mathbf{R}_y(\theta) \mathbf{R}_x(\phi)$$

순서가 바뀌면 다른 자세가 된다 — Yaw → Pitch → Roll 로 고정

제공받은 f_Trans_Body_To_Ned 는 인자를 (Roll, Yaw, Pitch) 로 받는다

구조체 필드 순서(Roll, Pitch, Yaw)와 달라 호출부마다 주석을 남겼다

01. 표적 구조체 설계

기동을 네 개의 값으로

"12 초부터 16 초까지 오른쪽으로 8 G" 라는 지시를 구조체 한 줄로 옮깁니다.

기동 1건 = 값 네 개

enTurnType 0 없음 / 1 Roll / 2 Yaw / 3 Pitch	EN_TurnType
gravityValue 크기는 세기, 부호는 회전 방향	FLOAT64 [G]
startTime 이 시각부터 (포함)	FLOAT64 [s]
endTime 이 시각까지 (제외)	FLOAT64 [s]

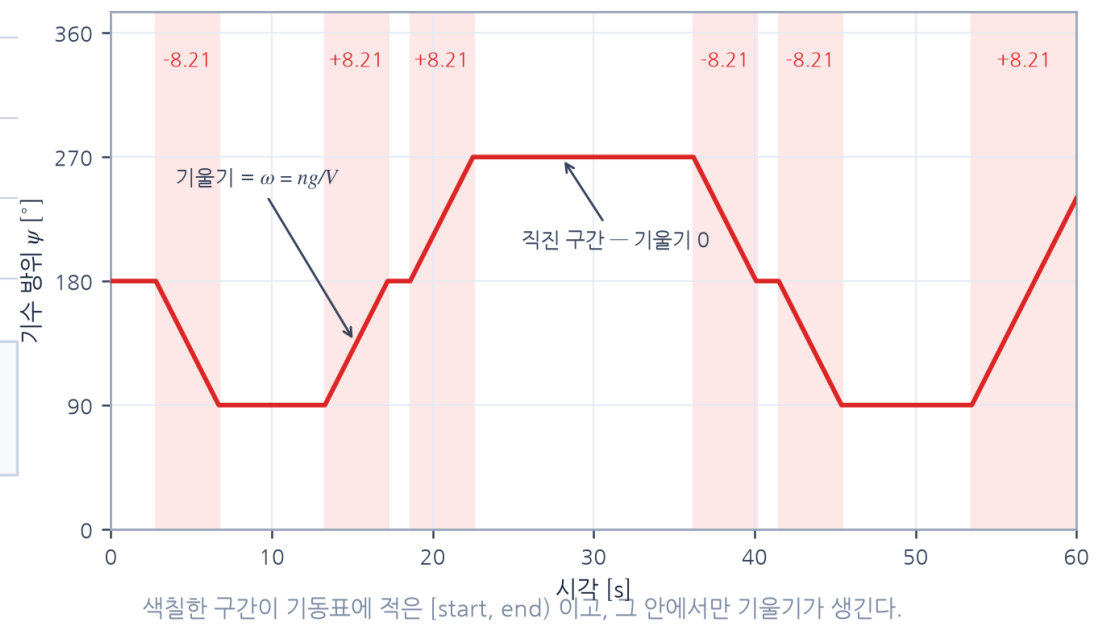
구간 규칙

$t \in [t_{start}, t_{end})$ 끝과 다음 시작이 같아도 겹침이 아니다
같은 표적에서 두 구간이 실제로 겹치면 TGT_ERR_MANEUVER_OVERLAP

표적 하나에 최대 30줄. 걸린 구간에서만 자세각이 돌고, 나머지 시간은 직진이다.

기동 지시는 보통 "위도 32.1 에서 좌 90도" 처럼 좌표로 온다. 구조체에는 시각 칸만 있으므로 Core 를 실제로 돌려 그 좌표에 닿는 시각을 찾아 start / end 로 옮겼다.

기동표 6줄이 실제 응답으로 나타난 모습 (대공 표적)

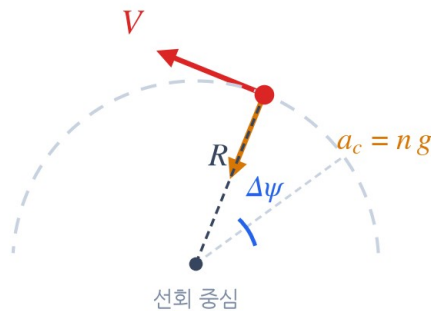


01. 표적 구조체 설계

G 값이 각속도와 선회 반경을 정한다

각속도를 직접 받지 않고 G 로 받는 이유는 , G 가 조종사와 기체가 실제로 견디는 한계이기 때문입니다 .

수평 선회의 기하



등속 원운동이므로 속도 V 는 접선 방향,
가속도 a_c 는 언제나 중심을 향한다.

이 그림 한 장에서 오른쪽 세 줄이 모두 나온다.

G 값에서 각속도와 선회 반경이 나온다

$$a_c = \frac{V^2}{R} = V\omega$$

원운동의 구심가속도

$$a_c = n g, \quad g = 9.80665 \text{ m/s}^2$$

기동 세기를 중력가속도의 n 배로 적는다

$$\omega = \frac{n g}{V}, \quad R = \frac{V^2}{n g}$$

G 값 하나가 회전 속도와 반경을 함께 정한다

코드에서는 한 줄이다 —

```
omega = (gravityValue * G_FORCE) / headingSpeed;
```

본 과제 두 표적에 G 를 걸면

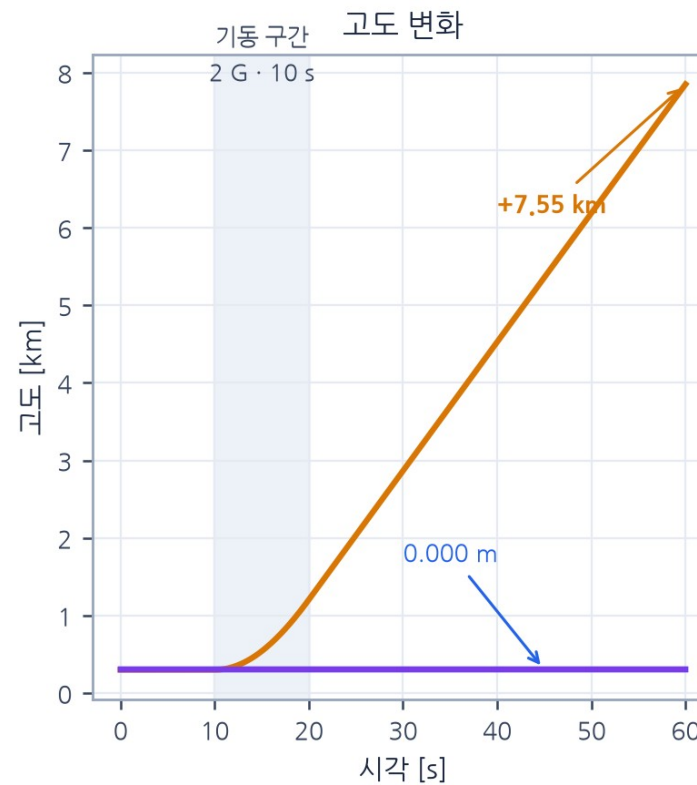
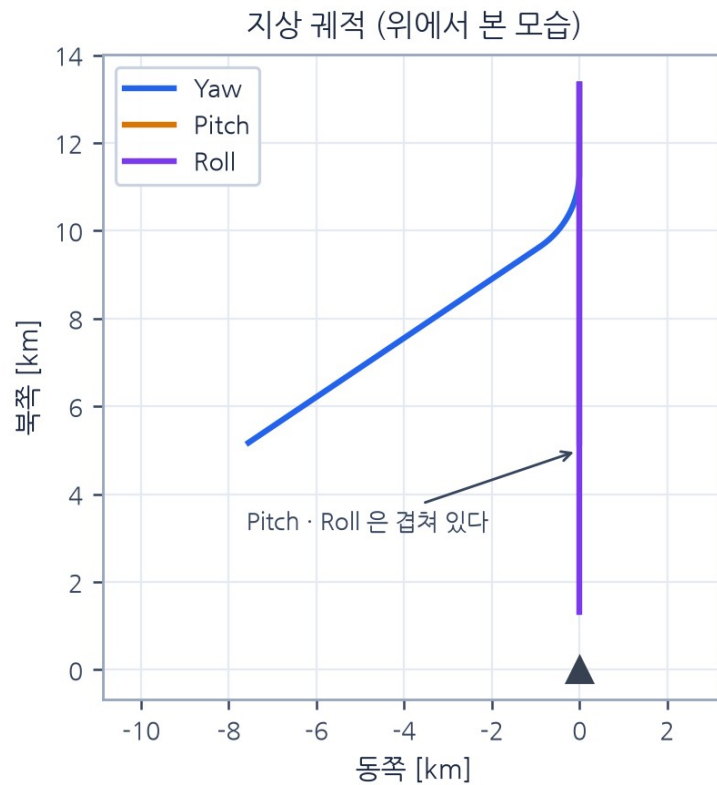
n [G]	대함 V = 30 m/s		대공 V = 200 m/s	
1	18.7 °/s	R = 91.8 m	2.8 °/s	R = 4079 m
2	37.5 °/s	R = 45.9 m	5.6 °/s	R = 2039 m
4	74.9 °/s	R = 22.9 m	11.2 °/s	R = 1020 m
8	149.8 °/s	R = 11.5 m	22.5 °/s	R = 510 m

같은 G 라도 빠른 표적은 덜 꺾인다 — 분모에 속력이 있기 때문이다.

01. 표적 구조체 설계

회전축 세 가지의 응답 — 실측

같은 표적에 같은 2 G 를 10 초씩, 축만 바꿔 걸어 본 결과입니다.



같은 표적 · 같은 2 G · 같은 10 s, 축만 바꿨다

Yaw

ψ 가 118° 돌고 궤적이 흰다.
고도는 300.000 m 그대로.

Pitch

θ 가 들려 7.55 km 상승.
지상 궤적은 거의 제자리.

Roll

ϕ 만 변하고 궤적은 불변.
회전축이 곧 속도 방향이라 그렇다.

Roll 이 궤적을 못 바꾸는 것은 구현 오류가 아니라 물리적으로 옳은 결과다.

01. 표적 구조체 설계

구조체 관계와 설계 결정

입력 — 시나리오가 채운다

ST_SimConfig			
durationTime	FLOAT64 [s]		
stepTime	FLOAT64 [s]		
st_Platform	ST_PlatformInit	1	
nTargetNum	INT32	1..10	
st_Target[10]	ST_TargetInit	10	

ST_TargetInit			
st_InitLla	STRUCT_Coord_Lla	rad, m	
st_InitAtt	STRUCT_Coord_Attitude	rad	
headingSpeed	FLOAT64 [m/s]	> 0	
nManeuverNum	INT32	0..30	
st_Maneuver[30]	ST_TargetManeuver	30	

ST_TargetManeuver			
enTurnType	EN_TurnType	0..3	
gravityValue	FLOAT64 [G]	±	
startTime / endTime	FLOAT64 [s]	[a, b)	

→
f_tgt_InitSim / f_tgt_StepSim

출력 — Core 가 채운다

ST_SimState			
nStepNum	INT32	600	
st_Sample	ST_SimSample	최근 1	
st_Config	ST_SimConfig	설정 사본	
설정을 복사해 두어 호출 중 원본이 바뀌어도 진행이 흔들리지 않는다			

ST_SimSample			
simTime	FLOAT64 [s]		
nStepIndex	INT32	0..600	
st_Platform	ST_TargetState	1	
st_Target[10]	ST_TargetState	10	

ST_TargetState			
st_PosEcef	STRUCT_Coord_Rect [m]	갱신	
st_VelEcef	STRUCT_Coord_Rect [m/s]	갱신	
st_Lla	STRUCT_Coord_Lla	출력용	
st_Att	STRUCT_Coord_Attitude	갱신	

설계 결정 세 가지

- 입력 / 출력 분리
- 고정 배열
- 제공 구조체 재사용

설정은 한 번만 읽고, 스텝마다 바뀌는 것은 상태뿐이다
 10 × 30 을 배열로 박아 실행 중 메모리 할당이 실패할 자리를 없앴다
 STRUCT_Coord_Lla / _Attitude 는 좌표변환 헤더의 것을 그대로 쓴다

01. 표적 구조체 설계

코드로 보는 구조체 ① — 입력

TargetSimCore/include/TargetSim.h

// 기동 회전축 . 값은 과제 명세를 그대로 따른다 .

typedef enum

```
{
    TGT_TURN_NONE = 0,    TGT_TURN_ROLL  = 1,
    TGT_TURN_YAW  = 2,    TGT_TURN_PITCH = 3
} EN_TurnType;
```

// 기동 1 건 . gravityValue 의 부호가 회전 방향이며

// 동체축 오른손 법칙을 따른다 .

typedef struct

```
{
    EN_TurnType    enTurnType;
    FLOAT64        gravityValue;    // [G] 중력가속도 배수
    FLOAT64        startTime;       // [s]
    FLOAT64        endTime;         // [s]
} ST_TargetManeuver;
```

// 위치는 위도 / 경도 / 고도 , 속도는 동체 x 축 방향 크기 .

typedef struct

```
{
    STRUCT_Coord_Lla    st_InitLla;    // Lat, Lon [rad] / Alt [m]
    STRUCT_Coord_Attitude st_InitAtt;   // Roll, Pitch, Yaw [rad]
    FLOAT64             headingSpeed;   // V_heading [m/s]
    INT32               nManeuverNum;
    ST_TargetManeuver    st_Maneuver[TGT_MAX_MANEUVER_NUM];
} ST_TargetInit;
```

구간은 [시작 , 종료)

한 기동의 끝과 다음 기동의 시작이 같아도 겹침으로 보지 않는다 . 경계를 열어 두어야 연속 기동을 한 줄씩 이어 붙일 수 있다 .

겹치면 계산 전에 거부

같은 표적에서 두 구간이 실제로 겹치면 어느 쪽을 따를지 정의되지 않는다 . TGT_ERR_MANEUVER_OVERLAP 을 돌려준다 .

10 × 30 은 고정 배열

구조체 하나가 약 10 KB. 동적 할당이 없어 실행 중 메모리 실패가 생길 자리가 없다 .

01. 표적 구조체 설계

코드로 보는 구조체 ② — 출력

TargetSimCore/include/TargetSim.h

```
// 한 시각의 상태 . 위치와 속도는 ECEF 에서 갱신하고 ,  
// LLA 는 출력과 다음 스텝의 NED 기준으로 함께 보관한다 .
```

```
typedef struct  
{  
    FLOAT64          simTime;      // [s]  
    STRUCT_Coord_Rect st_PosEcef;   // [m]  
    STRUCT_Coord_Rect st_VelEcef;   // [m/s]  
    STRUCT_Coord_Lla  st_Lla;  
    STRUCT_Coord_Attitude st_Att;  
} ST_TargetState;
```

```
// 한 시각의 플랫폼 · 표적 상태 . 호출자는 스텝마다  
// 이 구조체를 복사해 보관한다 .
```

```
typedef struct  
{  
    FLOAT64          simTime;  
    INT32            nStepIndex;  
    INT32            nTargetNum;  
    ST_TargetState    st_Platform;  
    ST_TargetState    st_Target[TGT_MAX_TARGET_NUM];  
} ST_SimSample;
```

ECEF 와 LLA 를 함께 두는 이유

- 갱신은 ECEF 에서 해야 곡률 향이 사라진다
 - 출력 · 화면 · CSV 는 LLA 라야 읽힌다
 - 다음 스텝의 속도를 만들 때도 그 지점의
 위도 · 경도가 필요하다
- 스텝당 한 번만 변환해 두면 세 용도를 모두 덮는다 .

플랫폼도 같은 구조체

플랫폼은 기동이 없는 표적일 뿐이다 .
같은 ST_TargetState 를 쓰고 각속도만 0 으로
넣어 같은 적분 경로를 통과시킨다 .

속력으로 나누는 연산은 기동 조회 안에만 있어 ,
정지 플랫폼 (V = 0) 도 안전하다 .

02

표적 궤적 모의 구현

Trajectory Propagation

- 표적 운동을 지배하는 방정식과 그 가정
- 동체 속도 한 개를 ECEF 속도 세 성분으로 — 과제의 추가 고민 항목
- 0.1 초를 전진시키는 방법과 오차 차수

02. 표적 궤적 모의 구현

표적 운동을 지배하는 세 줄

이 세 줄이 곧 구현이고, 아래 세 가지 가정을 명시적으로 두고 있습니다.

표적 운동을 지배하는 세 줄

$$\dot{\mathbf{p}}^e = \mathbf{v}^e$$

위치의 시간 변화율이 곧 속도다

$$\mathbf{v}^e = \mathbf{C}_n^e(\phi, \lambda) \mathbf{C}_b^n(\phi, \theta, \psi) [V \ 0 \ 0]^T$$

속도는 기수 방향 크기 V 를 두 번 돌려 만든다

$$\dot{\boldsymbol{\Theta}} = \boldsymbol{\omega}(t), \quad \omega_i = n_i g / V$$

자세각은 기동표가 지시하는 각속도로 돈다 (기동이 걸린 구간에서만 0 이 아니다)

이 모델이 전제하는 것

운동학 모델

힘이 아니라 속도 · 각속도를 직접 준다.

중력 · 추력 · 항력을 풀지 않으므로 탄도 표적에는 맞지 않는다.

속력 일정

회전만 하므로 $|\mathbf{v}|$ 는 변하지 않는다.

가속 · 감속이 필요하면 V 를 시간 함수로 바꿔야 한다.

오일러각 적분

자세각을 직접 적분한다.

$\theta \rightarrow \pm 90^\circ$ 에서 짐벌락 — 사원수로 바꾸면 풀린다.

02. 표적 궤적 모의 구현

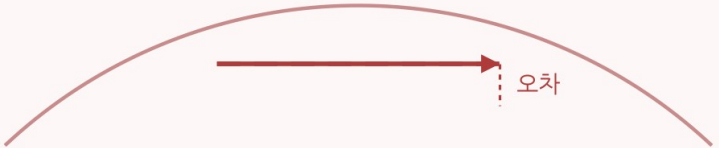
왜 ECEF 에서 적분하는가

명세가 정한 조건이지만, 왜 그래야 하는지는 오차 구조를 보면 분명합니다.

LLA 에서 직접 적분하면

$$\dot{\varphi} = \frac{v_N}{M(\varphi) + h}, \quad \dot{\lambda} = \frac{v_E}{(N(\varphi) + h)\cos\varphi}$$

분모의 곡률반경 M, N 이 위도마다 달라 매 스텝 다시 구해야 한다



지면(곡면)
곧게 날려도 스텝마다 고도가 $d^2/2R$ 씩 올라간다

오차

ECEF 에서 적분하면

$$\mathbf{p}_{k+1}^e = \mathbf{p}_k^e + \mathbf{v}^e \Delta t$$

미터 단위 직교좌표라 그냥 더하면 된다
지구가 둥근 것은 좌표계가 이미 품고 있다

대신 치러야 하는 값

- 사람이 못 읽는다 → 출력 때 LLA 로 되돌린다
- 속도를 만들 때는 그 지점의 위·경도가 필요하다

본 과제 조건에서 실제로 얼마인가

한 스텝 이동거리 $d = V\Delta t$

$200 \times 0.1 = 20 \text{ m}$

스텝당 고도 편차 $d^2/2R$

$31.36 \text{ } \mu\text{m}$

600 스텝 누적 (보정 없이)

18.81 mm

중점법으로 보정한 실측

고도 300.000000 m 고정

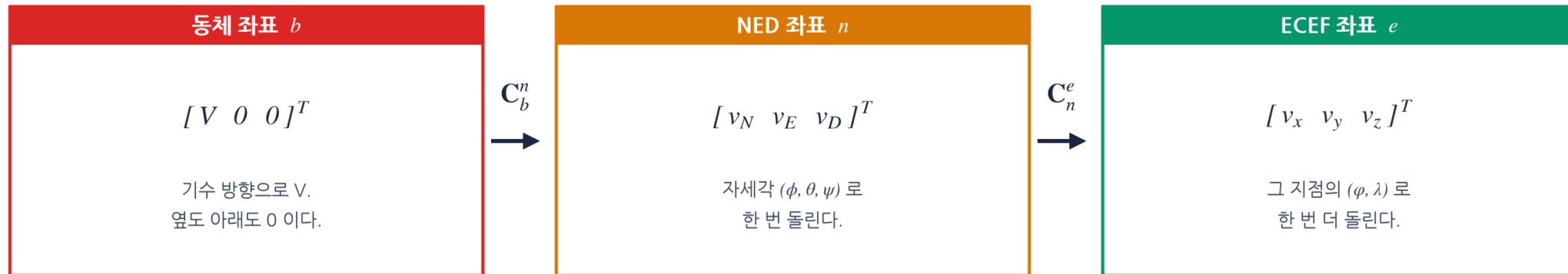
값 자체는 작지만, 이런 항이 남아 있으면 갱신 간격을 줄여도 오차가 선형으로만 줄어든다 — 2차 수렴이 깨진다.

02. 표적 궤적 모의 구현

동체 속도를 ECEF 속도로 — 과제의 추가 고민

" 표적 동체 속도 값 (V_heading) 을 ECEF 좌표계 Vx, Vy, Vz 성분으로 변환하는 방법 " 에 대한 답입니다 .

동체 속도를 ECEF 속도로 — 회전 두 번



$$\mathbf{v}^e = \mathbf{C}_n^e(\varphi, \lambda) \mathbf{C}_b^n(\phi, \theta, \psi) [V \ 0 \ 0]^T$$

두 번 다 회전행렬이다 — 직교행렬이므로 크기를 바꾸지 않는다

$$\|\mathbf{v}^e\| = \|\mathbf{C}_n^e \mathbf{C}_b^n [V \ 0 \ 0]^T\| = V$$

그래서 속력 보존은 증명된 성질이고, 시험은 구현 확인용이다

구현에서 주의한 두 가지

① 평행이동 제거

f_Trans_Ned_To_Ecef 는 회전에 더해 기준점만큼 평행이동까지 한다. 속도는 위치가 아니라 방향이므로 기준점 자리에 (0, 0, 0) 을 넣어 회전만 남겼다.

② 인자 순서

f_Trans_Body_To_Ned 는 (Roll, Yaw, Pitch) 순서로 받는다. 자세각 구조체 필드 순서(Roll, Pitch, Yaw)와 달라 호출부에 주석을 남겼다.

02. 표적 궤적 모의 구현

회전행렬의 원소

두 행렬을 펼치면 이렇게 됩니다. 위치에는 평행이동이 붙고, 속도에는 붙지 않습니다.

① 동체 → NED

$$\mathbf{C}_b^n(\phi, \theta, \psi)$$

$$\begin{bmatrix} c\psi c\theta & s\phi s\theta c\psi - c\phi s\psi & c\phi s\theta c\psi + s\phi s\psi \\ c\theta s\psi & s\phi s\theta s\psi + c\phi c\psi & c\phi s\theta s\psi - s\phi c\psi \\ -s\theta & s\phi c\theta & c\phi c\theta \end{bmatrix}$$

ϕ Roll θ Pitch ψ Yaw ($c = \cos$, $s = \sin$)

세 축 회전 $\mathbf{R}_z(\psi)\mathbf{R}_y(\theta)\mathbf{R}_x(\phi)$ 를 곱해 펼친 것이다.

② NED → ECEF

$$\mathbf{C}_n^e(L, \lambda)$$

$$\begin{bmatrix} -sL c\lambda & -s\lambda & -cL c\lambda \\ -sL s\lambda & c\lambda & -cL s\lambda \\ cL & 0 & -sL \end{bmatrix}$$

L 위도 λ 경도

표적이 움직이면 이 행렬도 매 스텝 다시 만들어야 한다.

두 행렬을 쓰는 방법 — 위치는 평행이동까지, 속도는 회전만

$$\mathbf{p}^e = \mathbf{C}_n^e \mathbf{p}^n + \mathbf{p}_{ref}^e$$

위치를 옮길 때는 기준점을 더한다

`f_Trans_Ned_To_Ecef(..., Pf_X, Pf_Y, Pf_Z, ...)`

$$\mathbf{v}^e = \mathbf{C}_n^e \mathbf{v}^n$$

속도는 방향뿐이라 기준점을 0 으로 준다

`f_Trans_Ned_To_Ecef(..., 0.0, 0.0, 0.0, ...)`

02. 표적 궤적 모의 구현

코드 ③ — 동체 속도를 ECEF 속도로

TargetSimCore/src/TargetSim.c

```
// 동체 속도 (V, 0, 0) 을 자세로 한 번 , 위경도로 한 번 돌려 ECEF 속도로 만든다 .
// 회전만 거치므로 속력은 그대로 보존된다 .
static STRUCT_Coord_Rect f_Tgt_GetVelEcef(const STRUCT_Coord_Attitude *st_Att,
    const STRUCT_Coord_Lla *st_Lla, FLOAT64 headingSpeed)
{
    STRUCT_Coord_Rect st_VelNed;

    // 참고 소스의 f_Trans_Body_To_Ned 는 인자를 Roll, Yaw, Pitch 순서로 받는다 .
    st_VelNed = f_Trans_Body_To_Ned(headingSpeed, 0.0, 0.0,
        st_Att->Roll, st_Att->Yaw, st_Att->Pitch);

    // f_Trans_Ned_To_Ecef 는 기준점 이동까지 더한다 . 방향만 옮길 때는 기준점을 원점으로 준다 .
    return f_Trans_Ned_To_Ecef(st_VelNed.x, st_VelNed.y, st_VelNed.z,
        0.0, 0.0, 0.0, st_Lla->Lat, st_Lla->Lon);
}
```

기준점을 0 으로 주는 이유

f_Trans_Ned_To_Ecef 는 회전에 더해 기준점만큼 평행이동까지 한다 . 속도는 "위치"가 아니라 "방향"이라 빼야 한다 .

인자 순서를 주석에 남긴 이유

제공받은 함수가 Roll, Yaw, Pitch 순서로 받는다 . 자세각 구조체 필드 순서와 달라 한 번 틀렸던 자리다 .

속력 보존은 성질 , 시험은 확인

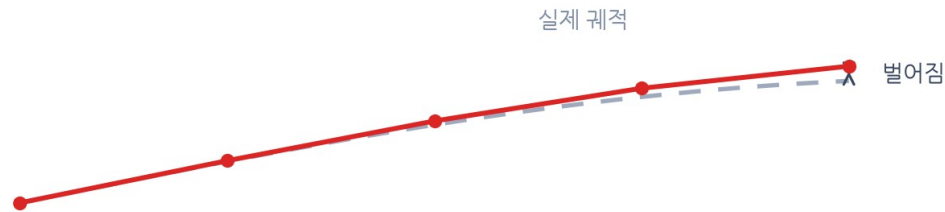
회전행렬은 직교행렬이라 크기를 바꾸지 않는다 . 601 표본에서 속력이 소수점 12 자리까지 입력값과 일치했다 .

02. 표적 궤적 모의 구현

0.1 초를 어떻게 전진시킬 것인가

속도가 계속 바뀌는 구간에서 어느 지점의 기울기를 쓰느냐가 오차 차수를 가릅니다.

오일러법 — 출발할 때의 기울기로 한 스텝 간다



중점법 — 반 스텝 가서 기울기를 다시 재고 한 스텝 간다



두 방법의 식과 오차 차수

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t f(t_k, \mathbf{x}_k)$$

국소 $O(\Delta t^2)$ · 전역 $O(\Delta t)$ — 1차

$$\mathbf{k}_l = f(t_k, \mathbf{x}_k)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t f\left(t_k + \frac{\Delta t}{2}, \mathbf{x}_k + \frac{\Delta t}{2} \mathbf{k}_l\right)$$

국소 $O(\Delta t^3)$ · 전역 $O(\Delta t^2)$ — 2차

이 과제에서의 반 스텝

자세각	각속도가 상수라 반 스텝 값을 바로 더한다
위치	스텝 시작 속도로 반 스텝 예측한다
속도	그 자리 · 그 자세에서 다시 만든다

계산량은 좌표변환 2회 → 4회로 늘지만, 0.1 s 에서
오차가 1.36 m → 1.36 cm 로 100 배 줄었다.

02. 표적 궤적 모의 구현

코드 ④ — 한 스텝 전진 (중점법)

TargetSimCore/src/TargetSim.c f_Tgt_Propagate (발췌)

```
halfStep = 0.5 * stepTime;

// ① 반 스텝 뒤의 자세 — 각속도가 구간 내 상수라 바로 더한다
st_AttMid.Roll = st_State->st_Att.Roll + (st_Rate->Roll * halfStep);
st_AttMid.Pitch = st_State->st_Att.Pitch + (st_Rate->Pitch * halfStep);
st_AttMid.Yaw = st_State->st_Att.Yaw + (st_Rate->Yaw * halfStep);

// ② 반 스텝 뒤의 위치 — 스텝 시작 속도로 예측
st_VelStart = f_Tgt_GetVelEcef(&st_State->st_Att, &st_State->st_Lla, headingSpeed);
st_PosMid.x = st_State->st_PosEcef.x + (st_VelStart.x * halfStep);
...

// ③ 그 자리 · 그 자세에서 구한 속도가 이 스텝의 대표 속도
st_LlaMid = f_Trans_Ecef_To_Lla(st_PosMid.x, st_PosMid.y, st_PosMid.z);
st_VelMid = f_Tgt_GetVelEcef(&st_AttMid, &st_LlaMid, headingSpeed);

// ④ 그 속도로 한 스텝을 옮긴다
st_Next.st_PosEcef.x = st_State->st_PosEcef.x + (st_VelMid.x * stepTime);
...
```

② 를 빼먹었을 때 생긴 일

처음에는 자세만 반 스텝 돌리고 위치는 그대로 두었다. 그러면 스텝 시작 지점의 위경도로 NED 를 잡게 되어, 표적은 앞으로 나아갔는데 "북쪽" 의 방향은 뒤에 남는다.

그 차이가 매 스텝 $d^2/2R$ 만큼 고도로 쌓인다.

알고도 남겨 둔 한계 — 짐벌락

오일러각을 직접 적분하므로 Pitch 가 $\pm 90^\circ$ 근처면 Yaw 축이 속도 방향과 겹쳐 Yaw 기동이 궤적을 바꾸지 못한다.

사원수로 바꾸면 풀리지만 과제 범위를 넘는다고 보고 코드에 주석으로 명시했다.

02. 표적 궤적 모의 구현

한 스텝 알고리즘과 계산 비용

f_Tgt_StepSim — 한 스텝에 벌어지는 일

for 각 객체 (플랫폼 1 + 표적 N)

① 현재 시각에 걸린 기동을 찾는다

rate = f_Tgt_GetAttRate(target, curTime)

② 자세각을 반 스텝 돌린다

attMid = att + rate * (dt / 2)

③ 위치를 반 스텝 예측한다

velStart = GetVelEcef(att, lla)

posMid = pos + velStart * (dt / 2)

llaMid = Ecef_To_Lla(posMid)

④ 그 자리 · 그 자세의 속도로 한 스텝 간다

velMid = GetVelEcef(attMid, llaMid)

posNew = pos + velMid * dt

⑤ 출력용 LLA 와 다음 속도를 만든다

llaNew = Ecef_To_Lla(posNew)

velNew = GetVelEcef(attNew, llaNew)

한 스텝의 비용

좌표변환 호출

4 회

GetVelEcef 2 + Ecef_To_Lla 2

행렬 곱셈

8 회

제공 라이브러리 Matrix_Product2

삼각함수

약 20 회

DCM 원소 계산

분기 · 나눗셈

기동 조회 1 회

속력으로 나누는 곳은 여기뿐

실측 성능

601 표본 × 객체 3

4.8 ms

한 스텝 · 한 객체

약 2.7 us

6001 표본 × 객체 11

0.5 s 미만

동적 할당 없음 · 스텝 간 상태 의존만 있음

④ 가 이 과제의 핵심이다

③ 에서 위치까지 반 스텝 옮기지 않으면

수평 비행 고도가 스텝마다 올라간다.

02. 표적 궤적 모의 구현

계산이 깨져도 상태는 지킨다

제공받은 변환 함수는 실패를 알려 주지 않습니다. 그래서 결과의 유한성을 직접 확인합니다.

계산이 깨져도 상태는 지킨다



그래서 생기지 않는 상태

- 표적 1 은 새 시각인데 표적 2 는 옛 시각인 표본
- 위치는 갱신됐는데 속도는 갱신 전인 객체

오류 코드 : NaN 이 섞인 채 다음 스텝으로 넘어가 전체가 오염되는 것

TGT_ERR_TARGET_NUM
표적 수 범위 밖
TGT_ERR_MANEUVER_OVERLAP
기동 구간 겹침

TGT_ERR_MANEUVER_NUM
기동 수 범위 밖
TGT_ERR_COORD
변환 결과가 비유한

TGT_ERR_TIME
시간 · 간격 · 기동 구간 이상
TGT_ERR_SIM_STATE
호출 사이 설정이 바뀜

TGT_ERR_SPEED
속력 0 이하 또는 비유한
TGT_ERR_SIM_END
마지막 스텝까지 진행 완료

03

시뮬레이션 결과 및 검증

Results and Verification

- 명세 조건 그대로 60 초를 돌린 결과
- 해석해 대조 · 불변량 보존 · 수렴 차수 — 세 가지 검증
- 기동을 엮은 시연과 운용 화면

03. 시뮬레이션 결과 및 검증

시뮬레이션 조건

「과제 3 내용」 2 항의 값을 그대로 넣었습니다.

	위도 [°]	경도 [°]	고도 [m]	속력 [m/s]	Roll [°]	Pitch [°]	Yaw [°]
플랫폼	32.0	126.0	0	0	0	0	0
대함 표적	32.125	126.03	0	30	0	0	270 (서)
대공 표적	32.12	126.0	300	200	0	0	180 (남)

시간 60 s · 간격 0.1 s

스텝 600 회, 표본 601 개.
t=0 과 t=60 을 모두 포함한다.
시각은 누적이 아니라 스텝 번호 × 간격으로
매겨 반올림 오차가 쌓이지 않게 했다.

검증하기 좋은 조건이다

기동이 없으므로 답을 미리 알 수 있다.
대함은 정서진 → 위도 불변,
대공은 정남진 → 경도 불변,
둘 다 수평 → 고도 불변.

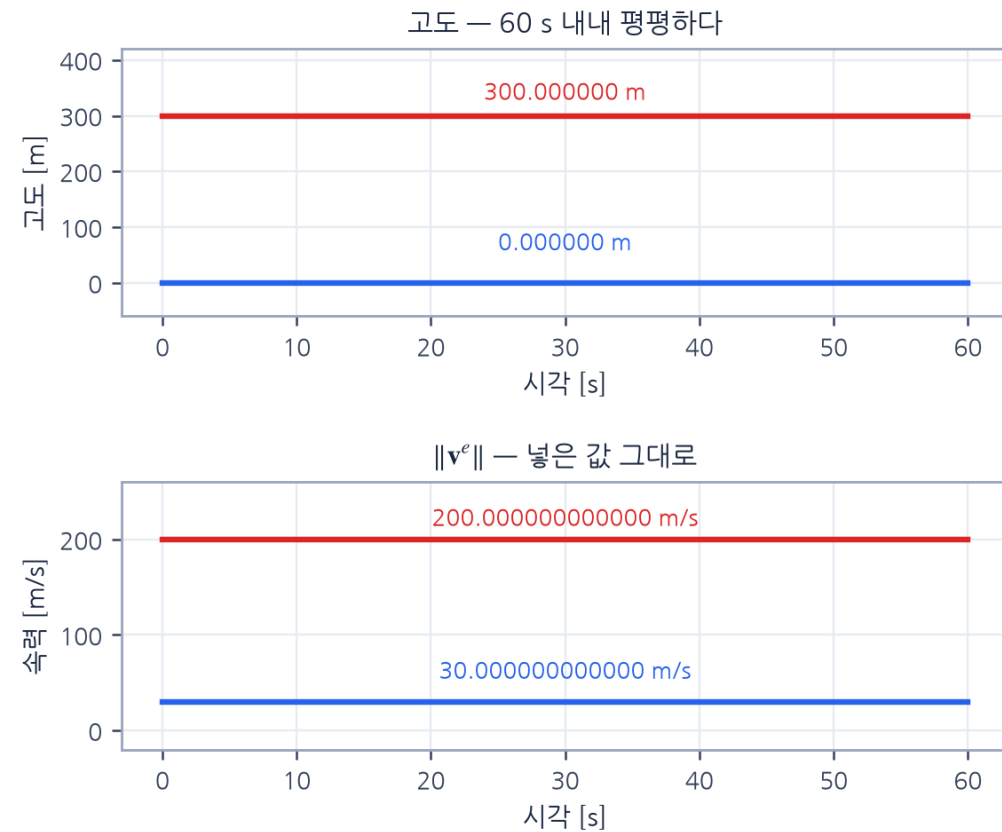
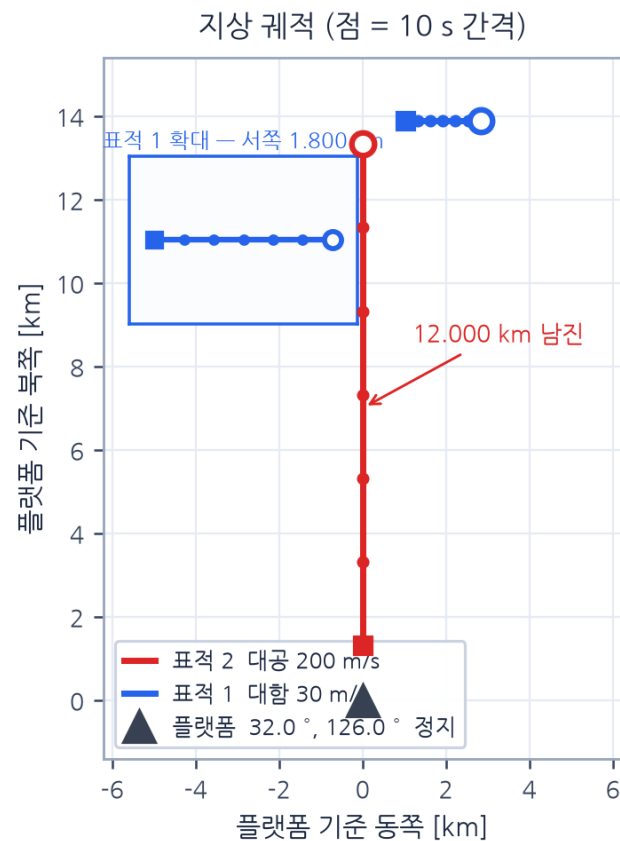
플랫폼 속력이 0 인 점

각속도 계산에 속력이 분모로 들어가지만,
그 계산은 기동 조회 안에만 있다.
플랫폼은 기동이 없어 그 자리를 지나가지 않는다.

03. 시뮬레이션 결과 및 검증

결과 ① — 60 초 궤적

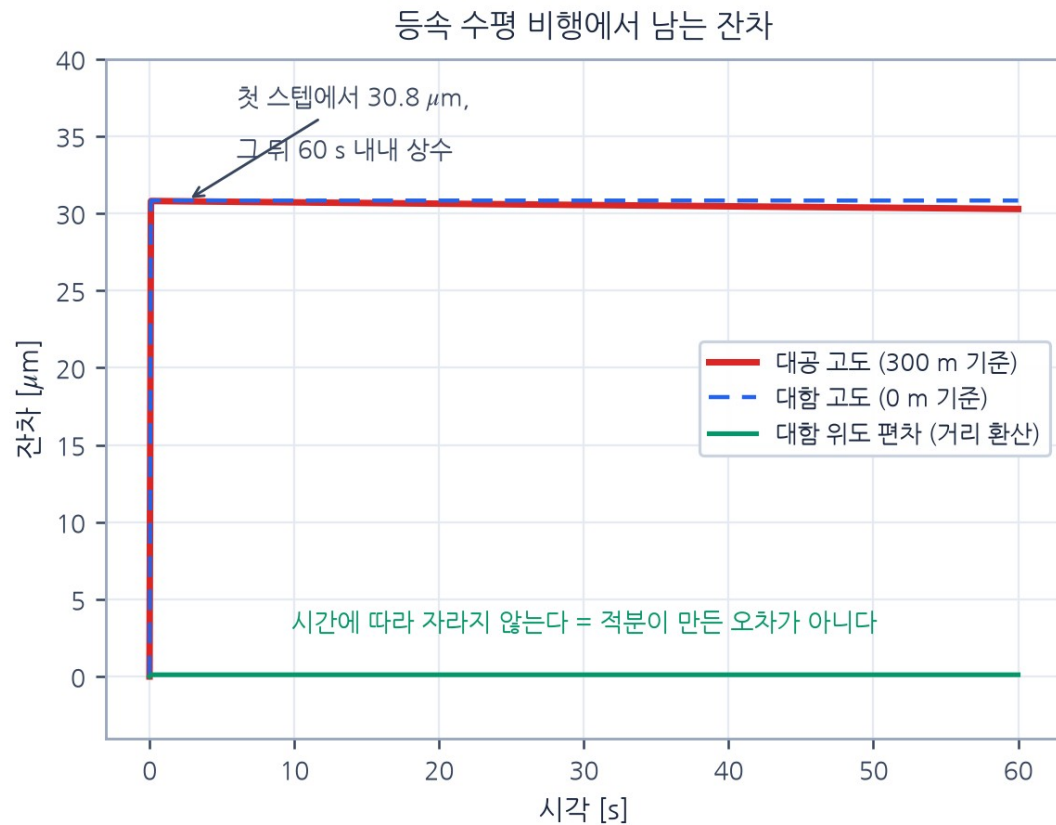
플랫폼을 원점으로 잡은 동 - 북 평면입니다. 점 하나가 10 초입니다.



03. 시뮬레이션 결과 및 검증

결과 ② — 불변량 잔차

변하면 안 되는 값이 실제로 얼마나 변했는지, 마이크로미터 단위까지 봤습니다.



불변량별 최대 잔차 (601 표본)

대공 고도	$30.8\ \mu\text{m}$
제공 코드의 ECEF→LLA 5회 반복 근사	
대함 고도	$30.8\ \mu\text{m}$
같은 원인 — 두 표적이 같은 값	
대함 위도	$0.11\ \mu\text{m}$
정서진 — 위도가 변하면 안 된다	
대공 경도	$0.00\ \mu\text{m}$
경남진 — 경도가 변하면 안 된다	
대공 속도	$0\ \text{m/s}$ (완전 일치)
회전행렬이 크기를 보존한다	
대함 속도	$0\ \text{m/s}$ (완전 일치)
601 표본 모두 비트까지 같다	

잔차의 출처는 적분기가 아니라 제공받은 좌표변환의 반복 근사다.
Bowring 폐형식으로 바꾸면 사라지지만, 과제 조건이 "제공 코드 사용" 이라 그대로 두었다.

03. 시뮬레이션 결과 및 검증

결과 ③ — 검증 종합

세 가지 방법으로 검증했다

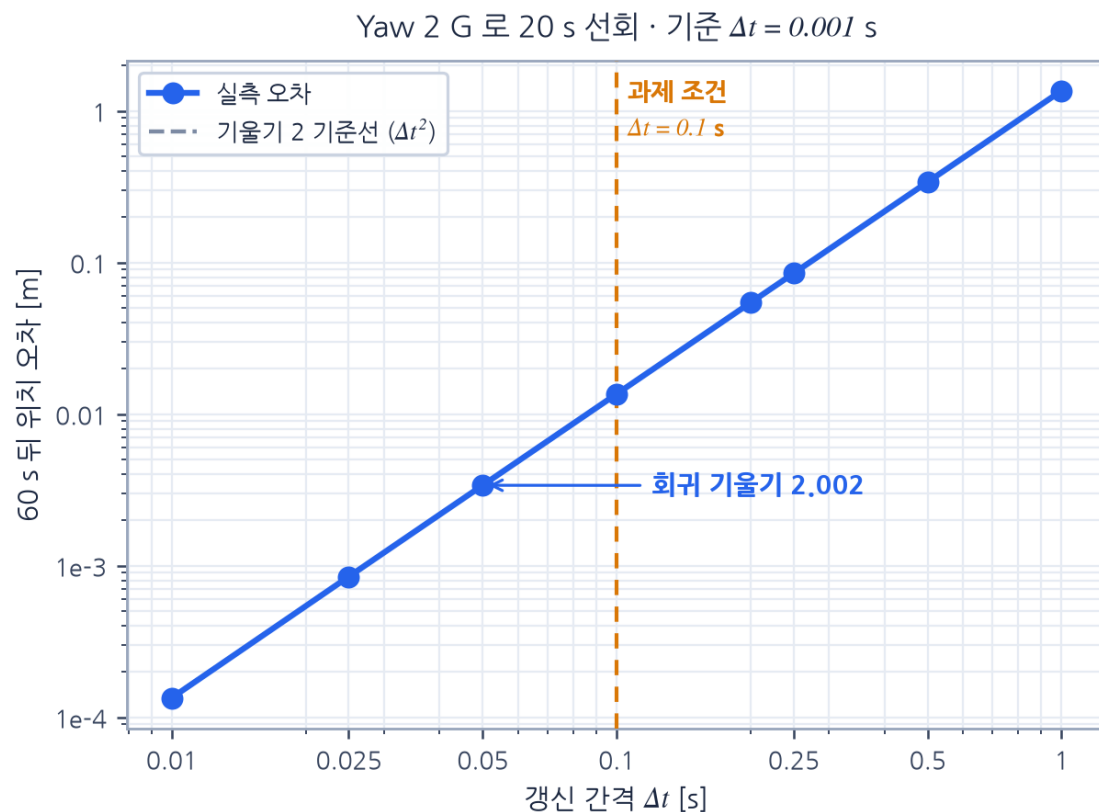
① 해석해와 대조	이론값 · 기대값	시뮬레이션 결과	잔차
대함 이동거리	$30 \times 60 = 1,800 \text{ m}$	1800.00000 m	+1.25e-08 m
대공 이동거리	$200 \times 60 = 12,000 \text{ m}$	12000.00000 m	+1.66e-07 m
② 불변량 보존	이론값 · 기대값	시뮬레이션 결과	잔차
대공 고도	300 m 유지	300.000000 ~ 300.000031 m	30.8 μm
대공 속력	200 m/s 유지	200.000000000000 m/s	0.0e+00
대함 위도	32.125 ° 유지 (정서진)	32.125000000000 °	0.1 μm
대공 경도	126.0 ° 유지 (정남진)	126.000000000000 °	0
③ 수렴 차수	이론값 · 기대값	시뮬레이션 결과	잔차
Δt 를 1/2 로	오차 1/4 (2차)	1.357 $\rightarrow$ 0.339 $\rightarrow$ 0.085 m	기울기 2.00
$\Delta t = 0.1 \text{ s}$	— (설계값 확인)	60 s 뒤 위치오차 1.36 cm	분해능 대비 무시

이동거리는 601개 표본 사이 거리를 전부 더해 낸 것이다 — 직선거리가 아니라 실제로 지나간 길이.
12 km 를 날아 잔차가 0.05 mm 면 적분기의 오차가 아니라 double 형의 표현 한계에 가깝다.

03. 시뮬레이션 결과 및 검증

결과 ④ — 갱신 간격에 대한 수렴

값 하나가 작은 것보다, 간격을 바꿨을 때 정해진 차수로 줄어드는 것이 더 강한 증거입니다.



읽는 법

두 선이 나란하다

간격을 절반으로 줄이면 오차가 1/4 로 줄어든다.
중점법이 설계대로 2차 정확도로 돌고 있다는 뜻.

$\Delta t = 0.1$ s 에서 1.36 cm

20 s 동안 2 G 로 꺾는 내내 쌓인 값이다.
레이다 거리 분해능(수 m)에 비하면 없는 값.

직진이면 더 작다

기동이 없으면 오차가 10 nm 수준 —
사실상 부동소수점 오차만 남는다.

차수가 확인의 핵심

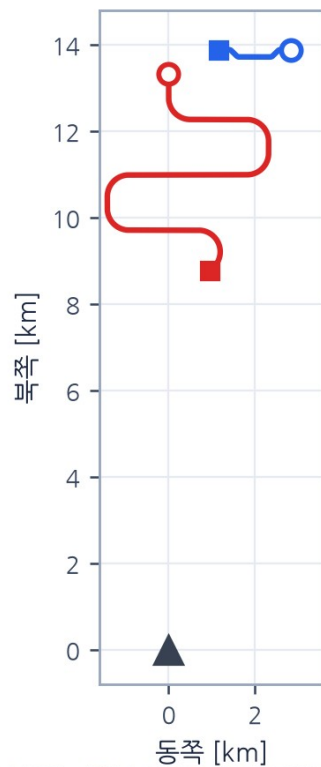
값 하나가 작은 것보다, 간격을 바꿨을 때
정해진 기울기로 줄어드는 것이 구현이 옳다는 증거다.

03. 시뮬레이션 결과 및 검증

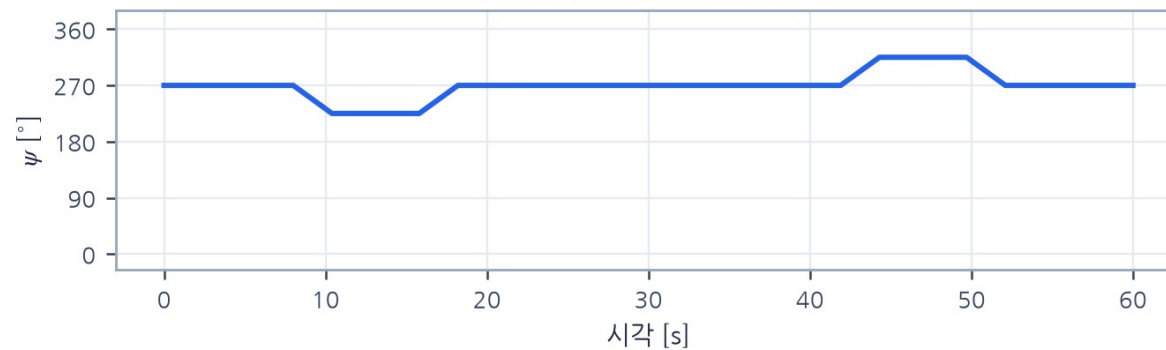
결과 ⑤ — 기동을 없으면

초기값 · 시뮬레이션 시간 · 갱신 간격은 명세 그대로, 기동표 10 줄만 추가했습니다.

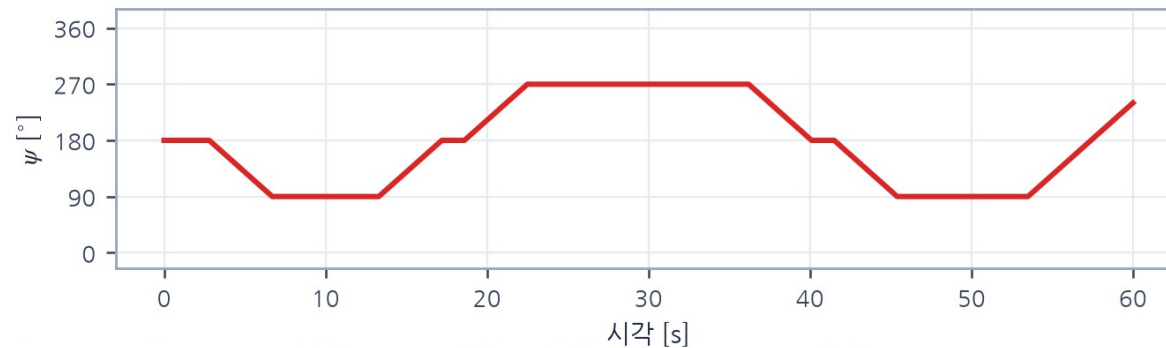
기동 10줄을 없앤 60 s 궤적



표적 1 대함 — 계단의 기울기가 곧 ω



표적 2 대공 — 계단의 기울기가 곧 ω

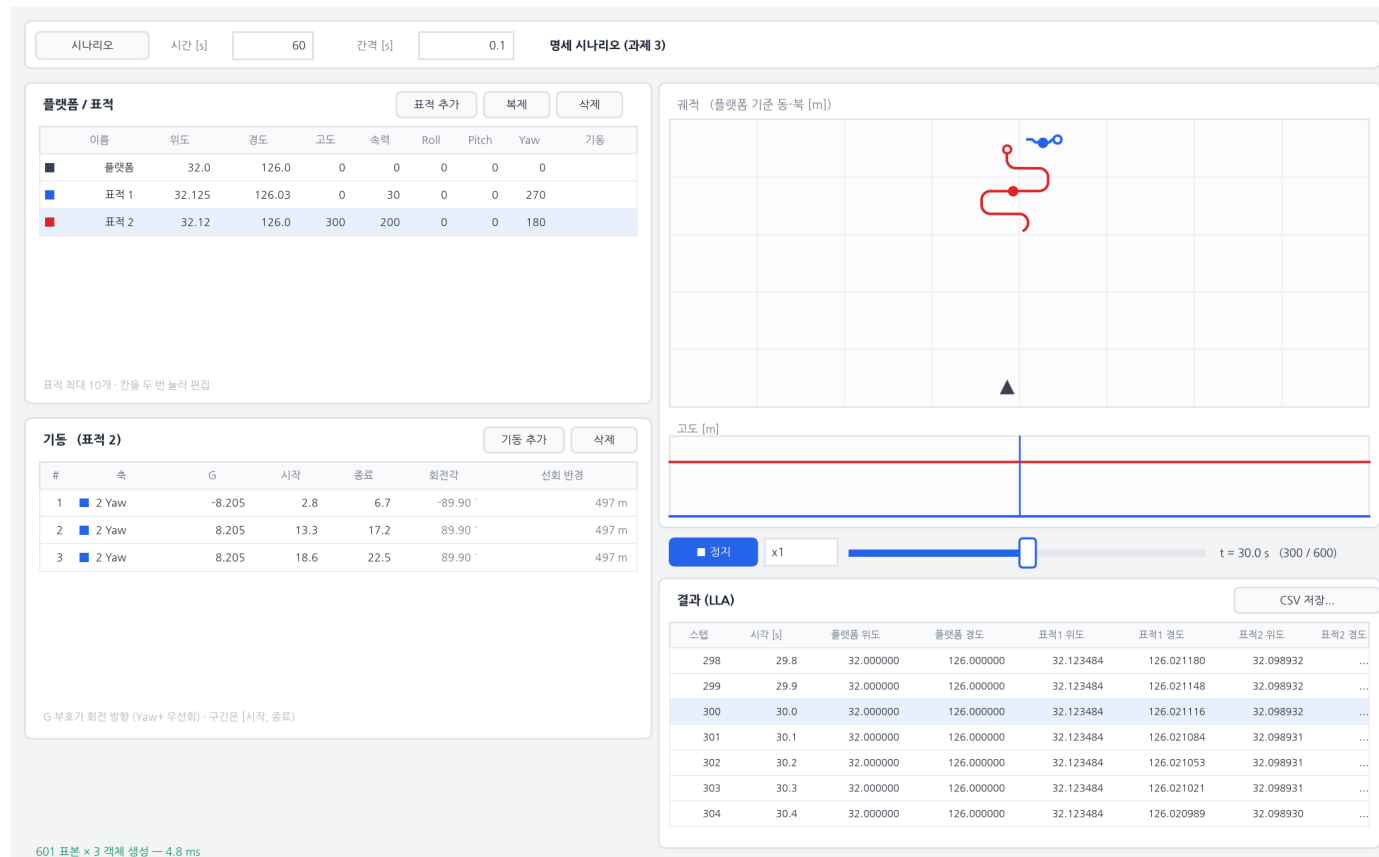


초기값 · 시뮬레이션 시간 · 갱신 간격은 명세 그대로다. 기동표 10줄만 추가했다. 회전 지점이 좌표로 주어져, Core 를 돌려 그 좌표에 닿는 시각을 찾아 start / end 로 옮겼다.

03. 시뮬레이션 결과 및 검증

TargetSim — 값을 바꿔 가며 확인하는 화면

표에서 숫자를 고치면 자동으로 다시 돌고, 그림에서 시작점과 기수를 끌 수 있습니다.



고치면 곧바로 다시 돈다

표의 칸을 고치면 0.25 초 뒤 자동으로 다시 계산한다. 601 표본 × 3 객체가 4.8 ms 라 기다릴 일이 없다.

그림에서 끌어서 배치

시작점과 기수 손잡이를 마우스로 끌면 위도 · 경도 · Yaw 가 표에 반영된다. 회전각과 선회 반경은 G 로부터 계산해 보여 준다.

결과는 표 · CSV · 그림

시각별 LLA 를 표로 보고, 슬라이더로 되감고, CSV 로 내보낸다. 시나리오는 텍스트 파일로 저장해 다시 불러올 수 있다.

03. 시뮬레이션 결과 및 검증

정리

요구사항과 결과를 한 장에 놓고 보겠습니다.

과제 요구사항	구현	근거
표적 10 · 기동 30	고정 배열로 상한, 범위 밖은 오류 코드	범위 검사
위치 LLA · 속도 V_{heading}	ST_TargetInit, 방향은 자세각이 담당	검증 통과
기동 = $G \cdot \text{축} \cdot \text{시작} \cdot \text{종료}$	ST_TargetManeuver 4 필드, 구간 겹침 검사	검증 통과
ECEF 에서 상태 갱신	위치 · 속도 ECEF, 출력용 LLA 병행 보관	고도 잔차 30 μm
$V_{\text{heading}} \rightarrow V_x, V_y, V_z$ (추가 고민)	동체 $\rightarrow$ NED $\rightarrow$ ECEF, 회전 2 회	속력 완전 보존
제공 좌표변환 코드 사용	CoordinateTransform.c / matrixCalcLib.c 원본	무수정
결과를 LLA 로 출력하고 그림으로	표 · CSV · 궤적 · 고도 · 방위 그림	601 표본 출력

남겨 둔 한계

- Pitch $\pm 90^\circ$ 근처의 짐벌락 — 사원수로 전환하면 해소
- 힘을 풀지 않는 운동학 모델 — 탄도 표적에는 부적합
- 고도 잔차 30.8 μm 는 제공 코드의 ECEF $\rightarrow$ LLA 반복 근사

이어 붙일 수 있는 것

- 궤적에 측정 잡음을 얹으면 그대로 추적 필터 입력
- Part 2 의 좌표변환을 붙이면 안테나 기준 $R \cdot A_z \cdot E_i$
- 표본 생성기를 바꾸면 RCS · 도플러까지 확장 가능

감사합니다

질문 주시면 답변드리겠습니다